

pypomp: Inference for partially observed Markov process models in Python with JAX

DRAFT IN PROGRESS

Aaron J. Abkemeier* Jun Chen* Kevin Tan Jesse Wheeler
Bo Yang Kunyang He Jonathan Terhorst Aaron A. King
Edward L. Ionides *These authors contributed equally

Methods for fitting and evaluating partially observed Markov process (POMP) mechanistic models are useful in a variety of fields such as epidemiology, ecology, and finance, but they are limited by their computational demands. In response, we introduce **pypomp**, a high-performance Python package for statistical inference using POMP models. Built to handle complex stochastic dynamic systems, it implements plug-and-play algorithms for likelihood-based inference including sequential Monte Carlo and iterated filtering. By utilizing parallel processing on graphical processing unit (GPU) hardware, **pypomp** is able to run these algorithms faster and more efficiently than its predecessor, the R package **pomp**. Models that took months to fit and evaluate in **pomp** now take only days in **pypomp**, facilitating the use of previously impractical models. In addition, **pypomp** includes methods for performing gradient descent on differentiable particle filters using JAX, which can maximize the likelihood of a model higher than what is practically possible with iterated filtering alone. The package offers a comprehensive interface that streamlines the entire pipeline of model construction, fitting, and analysis, allowing practitioners to develop and test large, complex models with minimal implementation overhead.

1 Introduction

Partially Observable Markov Process (POMP) models, also known as state-space models or hidden Markov models, provide a flexible mechanistic framework for modeling time-series dynamic systems where the underlying state process is not directly observed. Methods such as particle filtering (Arulampalam et al. 2002) and iterated filtering (E. L. Ionides, Breto, and

King 2006; Edward L. Ionides et al. 2015) have been developed to perform maximum likelihood estimation and evaluation for POMP models. A defining strength of these methods is that they are performed by simulating from the process model instead of evaluating exact transition probabilities (a property known as plug-and-play (Bretó et al. 2009)), which makes them suitable for examining a wide range of models unconstrained by needs for mathematical convenience. Consequently, POMP models find extensive application in epidemiology (Mitchen et al. 2024; Fox et al. 2022; Wen et al. 2024), ecology (Auger-Méthé et al. 2021; Marino et al. 2019; Blackwood et al. 2013), finance (Bretó 2014), and other domains where mechanistic modeling is desired.

Thus far, the `pomp` package ecosystem in R has provided a basis for POMP modeling of several types. The `pomp` package is designed for standard POMP models, while its extension packages `panelPomp`, `spatpomp`, and `phyloPomp` further enhance its capabilities for panel, spatio-temporal, and phylodynamic data analysis, respectively (King, Nguyen, and Ionides 2016; Bretó et al. 2025; Asfaw et al. 2024).

While powerful, statistical inference for POMP models poses substantial computational challenges. To achieve a parameter estimate which converges to the maximum likelihood estimate (MLE), it is often necessary to run Iterated Filtering 2 (IF2) for hundreds of iterations, utilizing thousands of particles for each iteration, starting from dozens of initial parameter estimates. One full pass through a time series may involve hundreds of simulation steps per particle. The aforementioned R packages are built only to use central processing unit (CPU) devices, which are relatively slow for these methods even when running independent method calls in parallel. Consequently, these methods are bottlenecked by their runtime. It may then be necessary to compromise on the number of particles or iterations used, or to accept that some jobs – such as those that run on the scale of months – are simply infeasible. Even if a job can finish in an acceptable time, the latency from submitting a job to receiving the results can slow down the pace of iterative model development, and the monetary cost of running lengthy jobs on advanced hardware can add up.

As it turns out, POMP methods can be sped up considerably. Many of the processes involved in POMP methods are embarrassingly parallel, such as simulating the state process for each of thousands of particles, running the particle filter multiple times for the same parameter set, and running iterated filtering from multiple starting parameter sets. This last point is holds true especially when estimating a profile likelihood to construct a confidence interval as in E. L. Ionides et al. (2017). Graphics processing unit (GPU) devices are well-suited for embarrassingly parallel operations of this scale.

To address the need for plug-and-play POMP methods that leverage GPU acceleration, we introduce the package `pypomp` (Abkemeier et al. 2024). Briefly stated, it is a Python implementation of the core algorithms in `pomp` and `panelPomp`, but written such that they can be run on both GPUs and CPUs. We accomplish this by using JAX (Bradbury et al. 2018), a high-performance numerical computing library that supports hardware acceleration and automatic vectorization. [AA: Need to change "90 times faster" to something more specific after we more rigorously compare `pypomp` with `pomp`. In addition, I think we should include the @korevaar2020

data set fit in this paper, as it provides perhaps the strongest motivation for developing `pypomp`.] The resulting package runs methods up to 90 times faster on GPUs than `pomp` can on CPUs. Jobs that would take months in `pomp` can be completed in days in `pypomp`, enabling plug-and-play likelihood-based inference on data sets that would otherwise be intractably large. This is demonstrated in Section [AA: add this section], where we fit a panel POMP model to the Korevaar, Metcalf, and Grenfell (2020) England and Wales measles dataset, which contains over one million observations. While this would normally take over 5 months in `pomp`, `pypomp` can finish the job in 41 hours [AA: revise these numbers as appropriate].

The remainder of this paper is organized as follows. Section 2 demonstrates the mathematical notation for POMP models and their related implementation in `pypomp`. Section 3 introduces the embedded methodologies. Section 4 presents data analysis workflows and benchmarking results. ?@sec-discussion concludes with a discussion of future work.

2 POMP Models in `pypomp`

2.1 Model setup

Suppose we observe a process at discrete times $\{t_n; n = 1, \dots, N\}$, and the underlying process starts at time t_0 . Data $Y_{1:N}$ are modeled as noisy, incomplete, and indirect observations of a latent (unobserved) Markov process $\{X_{0:N}\}$ at the corresponding time.

A POMP model is defined by three building components, each corresponding to a user-supplied Python function in `pypomp`:

1. initial density: $f_{X_0}(x_0; \theta)$, implemented via `rinit`, generates draws of the initial states X_0 .
2. transition density: $f_{X_n|X_{n-1}}(x_n | x_{n-1}; \theta)$, implemented via `rproc`, propagates the evolution of latent states.
3. measurement density: $f_{Y_n|X_n}(y_n | x_n; \theta)$, implemented via `dmeas`, evaluates or simulates observations conditional on the latent states.

Here, we hold an conditional independence assumption that, given X_n , the observation Y_n is independent of all other variables. Then, the joint density of $(X_{0:N}, Y_{1:N})$ can be expressed as the product of the initial distribution, the transition densities, and the measurement densities:

$$f_{X_{0:N}, Y_{1:N}}(x_{0:N}, y_{1:N}; \theta) = f_{X_0}(x_0; \theta) \prod_{n=1}^N f_{X_n|X_{n-1}}(x_n | x_{n-1}; \theta) \prod_{n=1}^N f_{Y_n|X_n}(y_n | x_n; \theta)$$

The marginal likelihood of the observations is $\mathcal{L}(\theta) = f_{Y_{1:N}}(y_{1:N}; \theta) = \int f_{X_{0:N}, Y_{1:N}}(x_{0:N}, y_{1:N}; \theta) dx_{0:N}$. Such POMP models can include all the features desired by (Bjørnstad and Grenfell 2001). In practice, this integral is intractable for most nonlinear or non-Gaussian POMP models, motivating the use of simulation-based inference methods such as particle filtering.

2.2 Implementations of POMP models in pypomp

2.2.1 Object-oriented interface

A POMP model in `pypomp` is represented as an object of class `Pomp`, which encapsulates the model components: the initial state distribution, process model, and measurement model. This object-oriented interface allows users to specify by passing components to the constructor, including observations, model parameters, model mechanics such as simulators and the measurement density, covariates, and times. After the components are passed into the constructor, the constructor automatically generates additional internal elements, such as extended observations and covariates required for interpolation

Table 1 summarizes the main arguments to the `Pomp` constructor and their correspondence to mathematical objects.

Table 1: Main arguments to the `Pomp` class and related constructor objects.

Constructor	Attributes	Type	Description / Mathematical representation
Pomp	<code>rinit</code>	<code>RInit</code>	simulate initial states $X_0 \sim f_{X_0}(x_0; \theta)$
	<code>rproc</code>	<code>RProc</code>	simulate state transitions $X_n \sim f_{X_n X_{n-1}}(x_n x_{n-1}; \theta)$
	<code>rmeas</code>	<code>RMeas</code>	simulate observations $Y_n \sim f_{Y_n X_n}(y_n x_n; \theta)$
	<code>dmeas</code>	<code>DMeas</code>	evaluate measurement density $f_{Y_n X_n}(y_n x_n; \theta)$
	<code>ys</code>	<code>pandas.DataFrame</code>	observations $y_{1:N}^*$ with times $t_{1:N}$
	<code>covars</code>	<code>pandas.DataFrame</code>	covariates $z_{1:N}^*$ with times $s_{1:N}$
	<code>theta</code>	<code>dict</code>	parameters θ
	<code>par_trans</code>	<code>Partrans</code>	parameter transformation object
	<code>results_history</code>	<code>list</code>	History of the results for POMP methods run on the object
	<code>fresh_key</code>	<code>jax.Array</code>	A fresh, unused key is stored in the attribute after running a method on the object with a key argument

where `RInit`, `RProc`, `DMeas`, and `RMeas` are sub-constructors that specify the stochastic mechanisms of the model—the initial state simulator, state process transition, measurement density, and measurement simulator, respectively. The user provides each component as a plain Python function and `pypomp` then wraps into a JAX-compatible class instance. These wrapped components are passed as arguments when initializing the main `Pomp` object. The `ParTrans`

sub-constructor defines parameter transformations between the specific estimation space and the natural parameter space.

We demonstrate here how to create a `Pomp` object. Specifically, we show how to create the linear Gaussian model included in the package as `LG()`. We begin by importing necessary packages and defining helper functions for handling the parameters. Because `pypomp` will run our defined model components within JAX JIT-compiled code, it is necessary to write the components to be JAX-compliant. Naturally, the JAX package has many useful functions for this purpose. We also generate a pseudorandom number generation (PRNG) key to be used with JAX random number generators. All stochastic simulations in `pypomp` are controlled via JAX PRNG keys, ensuring full reproducibility when using the same seed.

```
import pypomp as pp
import pandas as pd
import jax
import jax.numpy as jnp
from functools import partial

def get_thetas(theta):
    A = jnp.array(
        [theta["A11"], theta["A12"], theta["A21"], theta["A22"]]
    ).reshape(2, 2)
    C = jnp.array(
        [theta["C11"], theta["C12"], theta["C21"], theta["C22"]]
    ).reshape(2, 2)
    Q = jnp.array(
        [theta["Q11"], theta["Q12"], theta["Q21"], theta["Q22"]]
    ).reshape(2, 2)
    R = jnp.array(
        [theta["R11"], theta["R12"], theta["R21"], theta["R22"]]
    ).reshape(2, 2)
    return A, C, Q, R

def transform_thetas(A, C, Q, R):
    return jnp.concatenate([A.flatten(), C.flatten(), Q.flatten(), R.flatten()])

rw_sd = pp.RWSigma(
    sigmas={
        "A11": 0.02, "A12": 0.02,
        "A21": 0.02, "A22": 0.02,
```

```

    "C11": 0.02, "C12": 0.02,
    "C21": 0.02, "C22": 0.02,
    "Q11": 0.02, "Q12": 0.02,
    "Q21": 0.02, "Q22": 0.02,
    "R11": 0.02, "R12": 0.02,
    "R21": 0.02, "R22": 0.02,
},
init_names=[],
)

key = jax.random.key(1)

```

2.2.2 Model Components

We refer to model components describing initialization, transfer, or measurement processes as the model mechanics, including `rinit`, `rproc`, `dmeas`, and `rmeas`. Users must define these mechanics as Python functions. Specifically, we require users to provide function code to the object constructor, which verifies that all necessary function arguments are included and in the correct order. This requirement stems from `pypomp`'s internal mechanism: it vectorizes component functions using `jax.vmap()` to efficiently run thousands of particles. Since `jax.vmap()` maps functions to input arrays by position rather than keyword, users must strictly adhere to parameter order. While all expected arguments must be included, the function does not need to utilize all of them. We include here an example defining the model mechanics for the linear gaussian model. In practice, at least one of `dmeas` or `rmeas` must be provided, while definitions for `rinit` and `rproc` are always required.

```

def rinit(theta_, key, covars=None, t0=None):
    """Initial state process simulator for the linear Gaussian model"""
    A, C, Q, R = get_thetas(theta_)
    result = jax.random.multivariate_normal(key=key, mean=jnp.array([0, 0]), cov=Q)
    return {"X1": result[0], "X2": result[1]}

def rproc(X_, theta_, key, covars=None, t=None, dt=None):
    """Process simulator for the linear Gaussian model"""
    A, C, Q, R = get_thetas(theta_)
    X_array = jnp.array([X_["X1"], X_["X2"]])
    result = jax.random.multivariate_normal(key=key, mean=A @ X_array, cov=Q)
    return {"X1": result[0], "X2": result[1]}

```

```

def dmeas(Y_, X_, theta_, covars=None, t=None):
    """Measurement model distribution for the linear Gaussian model"""
    A, C, Q, R = get_thetas(theta_)
    X_array = jnp.array([X_["X1"], X_["X2"]])
    Y_array = jnp.array([Y_["Y1"], Y_["Y2"]])
    return jax.scipy.stats.multivariate_normal.logpdf(Y_array, X_array, R)

def rmeas(X_, theta_, key, covars=None, t=None):
    """Measurement simulator for the linear Gaussian model"""
    A, C, Q, R = get_thetas(theta_)
    X_array = jnp.array([X_["X1"], X_["X2"]])
    return jax.random.multivariate_normal(key=key, mean=C @ X_array, cov=R)

```

2.2.3 Parameters

The `Pomp` constructor also requires model parameters. These can be provided either as a dictionary or as a list of dictionaries. Each item in a dictionary should include the parameter name as the key and the parameter value as the dictionary value.

If the parameter sets are provided as a list of dictionaries, methods such as `pfilter()` run on each set of parameters. Here, we use `Pomp.sample_params()` to sample sets of parameters from uniform distributions with bounds passed as a dictionary of length-2 tuples. `Pomp.sample_params()` returns a ready-to-use list of dictionaries with the sampled parameters.

```

theta = {
    "A11": jnp.cos(0.2), "A12": -jnp.sin(0.2),
    "A21": jnp.sin(0.2), "A22": jnp.cos(0.2),
    "C11": 1.0,          "C12": 0.0,
    "C21": 0.0,          "C22": 1.0,
    "Q11": 0.01,         "Q12": 1e-6,
    "Q21": 1e-6,         "Q22": 0.01,
    "R11": 0.1,          "R12": 0.01,
    "R21": 0.01,         "R22": 0.1,
}
param_bounds = {k: (v * 0.9, v * 1.1) for k, v in theta.items()}
n = 5
key = jax.random.key(1)
key, subkey = jax.random.split(key)
theta_list = pp.Pomp.sample_params(param_bounds, n, subkey)

```

2.3 Covariates

POMP models often involve external time-varying inputs, referred to as covariates, which can influence either the latent process or the measurement model. Examples include seasonality, interventions, or environmental drivers in ecological applications. In `pypomp`, covariates are supplied as a `pandas.DataFrame` indexed by time. The covariate time points may differ from the observation times, so the `Pomp` object automatically interpolates covariate values at the observation times. The linear gaussian model does not involve any covariates, and an example using covariates is given in the Data Analysis Section.

2.3.1 POMP Object Construction

We do not have real data in this LG example, so we generate our own. To make this example cleaner, we use the function `LG()` to construct the completed linear Gaussian model object and then generate the data using `simulate()`. Observation times are provided to the `Pomp` constructor via the `pandas.DataFrame` row index. If covariates were provided, the times at which the covariates were observed would also be provided by the `pandas.DataFrame` row index.

```
T = 100
# ensure `key` exists; if not, uncomment the next line
# key = jax.random.PRNGKey(1)

key, subkey = jax.random.split(key)
_, ys = pp.models.LG(T=T).simulate(key=subkey)
ys = ys.rename(columns={"obs_0": "Y1", "obs_1": "Y2"})
ys = ys[["time", "Y1", "Y2"]]
ys.set_index("time", inplace=True)

LG_obj = pp.Pomp(
    rinit=rinit,
    rproc=rproc,
    dmeas=dmeas,
    rmeas=rmeas,
    t0=0.0,
    nstep=1,
    ys=ys,
    theta=theta_list,
    statenames=["X1", "X2"],
)

print("LG_obj created; ys.shape =", ys.shape)
```


W0513 16:15:20.037264 4626633 cpp_gen_intrinsics.cc:74] Empty bitcode string provided for ei

```
LG_obj created; ys.shape = (100, 2)
```

Each argument to `Pomp` is accessible from the object as an attribute.

```
print(LG_obj.rinit) # access POMP model components
print(LG_obj.rproc)
print(LG_obj.dmeas)
print(LG_obj.rmeas)
print(LG_obj.theta) # access parameters
print(LG_obj.ys.head()) # access observations
```

```
<pypomp.core.model_struct._RInit object at 0x12f68cdd0>
<pypomp.core.model_struct._RProc object at 0x12f64f170>
<pypomp.core.model_struct._DMeas object at 0x12ade3e90>
<pypomp.core.model_struct._RMeas object at 0x12ade0e30>
PompParameters(n_replicates=5, n_params=16)
      Y1      Y2
time
1.0  -0.115930  0.724815
2.0  -0.353621  0.230780
3.0   0.079373 -0.154176
4.0   0.148403 -0.297087
5.0   0.655207  0.529257
```

2.3.2 Premade models:

Beyond the linear gaussian model, `pypomp` includes several ready-to-use model constructors that serve as examples for what users can do:

1. `LG()` — a simple linear-Gaussian model with 2-dimensional latent and observed states; useful to validate API usage and diagnostics.
2. `spx()` — the S&P500 log-return model from Sun et al. (Sun 2024).
3. `dacca()` — the cholera transmission model from King et al. (King et al. 2008).
4. `UKMeasles.Pomp()` — the measles district model from He et al. (He, Ionides, and King 2010), extended to the Korevaar et al. dataset (Korevaar, Metcalf, and Grenfell 2020). Panel and spatial variants (PanelPOMP/SpatPOMP style) are planned.

These examples demonstrate correct model specification. If a user's model throws an error or runs slowly, they may compare its components to a premade model to make improvements.

2.3.3 Panel POMP class

3 POMP Methods in pypomp

In this section, we describe the core elementary and inference methods currently implemented in `pypomp`, including:

- **Particle Filter** (Sequential Monte Carlo, written in `pfilter()`): A standard sequential Monte Carlo algorithm for likelihood evaluation and (optional) state estimation, forming the basis for most inference methods in POMP models.
- **Differentiated Measurement-off-policy Particle Filter** ($\text{DMOP}(\alpha)$, written in `mop()`): A recently proposed SMC method (Tan, Hooker, and Ionides 2024) that evaluates the likelihood at one parameter value while obtaining resampling decisions from another, adjusting via discounted off-parameter measurement weights.
- **Iterated Filtering** (IF2, `mif()`): A classical IF2 algorithm (Edward L. Ionides et al. 2015) for likelihood-based parameter inference that maximizes the likelihood via particle filtering.
- **Iterated Filtering with Automatic Differentiation** (`train()`): A recently proposed AD-based algorithm (Tan, Hooker, and Ionides 2024) that leverages $\text{DMOP}(\alpha)$ to perform maximum likelihood estimation with gradient descent methods.

It also contains other class functions for better model diagnostics after running the above elementary and inference methods:

- `simulate()` performs simulations of the POMP model, i.e. it samples from joint distributions of latent states and observations.
- `traces()` returns the full trace of log-likelihoods and parameters from the entire result history.
- `results(idx)` returns the results of the method run at the given index.
- `time()` summarizes the execution times of methods run.
- `prune()` replaces `self.theta` with a list of top n thetas based on the most recent available log-likelihood estimates.
- `plot_traces()` generates the plots containing parameter and log-likelihood traces from the entire result history.
- `print_summary()` prints a summary of the POMP object.

A key feature of the above POMP inference methods lies in the **plug-and-play property** (E. L. Ionides, Breto, and King 2006), meaning that inference algorithms can be implemented without requiring explicit evaluation of the transition density of the latent process. Instead, it suffices for the user to provide a the latent process simulator (`rproc`), the initial state simulator (`rinit`), and the observation density (`dmeas`). This property enables POMP methods to be widely applied to complex mechanistic models where transition densities are intractable.

In `pypomp`, the plug-and-play design is fully preserved: users only need to provide component functions compliant with JAX requirements, which can be directly plugged in inference methods like `pfilter()`, `mop()`, `mif()`, and `train()`. The package combines the generality of plug-and-play modeling with the efficiency of JAX compilation and vectorization.

Unlike the R family of POMP packages, some `Pomp` class methods including `pfilter()`, `mif()` and `train()` yield results by modifying the object in place instead of returning new objects. All of the results are stored a list under `LG_obj.results_history`, which is an attribute under `Pomp` class object `LG_obj`. Each element in the list corresponds to one method call. Each element includes results such as the log-likelihood and parameter estimates when applicable as well as the inputs used for the function call, so it is easy to keep track of how the results were calculated. If multiple parameter sets are supplied in a list as an argument, the method evaluates at each set and the results for each are stored. Meanwhile, each following method in `pypomp` that takes a key as an argument stores an unused child key under `LG_obj.fresh_key` that later method calls can use by default when a key argument is not given. This design prevents repeated use of the same key across a sequence of method calls and ensures proper randomness.

```
LG_obj.pfilter(J=100, reps=5, key=subkey)
LG_obj.mif(rw_sd=rw_sd, M=2, a=0.5, J=100, key=subkey)

print(LG_obj.results_history.traces())
```

	theta_idx	iteration	method	logLik	A11	A12	A21	\
0	0	0	pfilter	-99.785549	0.963512	-0.178802	0.203705	
1	0	0	mif	NaN	0.963512	-0.178802	0.203705	
2	0	1	mif	-140.500534	0.827827	-0.039467	0.233428	
3	0	2	mif	-146.522827	0.831906	0.023373	0.003930	
4	1	0	pfilter	-90.157137	0.966490	-0.178802	0.214532	
5	1	0	mif	NaN	0.966490	-0.178802	0.214531	
6	1	1	mif	-151.566376	0.928734	-0.193895	-0.049925	
7	1	2	mif	-155.627869	0.815899	0.014328	-0.009548	
8	2	0	pfilter	-90.953700	0.967444	-0.178802	0.185848	
9	2	0	mif	NaN	0.967444	-0.178802	0.185848	
10	2	1	mif	-137.099670	0.942031	-0.222330	-0.004752	
11	2	2	mif	-136.200531	0.624619	-0.250220	0.067008	
12	3	0	pfilter	-104.294093	1.015698	-0.178802	0.192220	
13	3	0	mif	NaN	1.015698	-0.178802	0.192220	
14	3	1	mif	-143.689774	1.034310	-0.245711	0.156650	
15	3	2	mif	-134.575409	0.800505	-0.243012	0.115533	
16	4	0	pfilter	-88.790386	0.977908	-0.178802	0.195180	
17	4	0	mif	NaN	0.977909	-0.178802	0.195180	
18	4	1	mif	-140.670212	0.786681	-0.161761	0.244725	

19 4 2 mif -136.936722 0.773738 -0.179715 0.297137

	A22	C11	C12	C21	C22	Q11	Q12 \
0	1.069749	0.999112	0.000000	0.000000	1.054883	0.010472	9.363539e-07
1	1.069749	0.999111	0.000000	0.000000	1.054883	0.010472	9.363539e-07
2	0.929862	1.125984	-0.129401	0.024574	1.036649	0.104627	1.631406e-01
3	0.998267	1.349301	-0.372748	0.097255	0.889322	0.176813	2.261432e-01
4	0.982279	0.941170	0.000000	0.000000	1.077328	0.009964	1.079856e-06
5	0.982279	0.941170	0.000000	0.000000	1.077328	0.009964	1.079856e-06
6	0.930576	0.984762	0.090999	-0.029913	0.880255	0.143922	-1.585881e-01
7	0.552129	0.869279	0.175711	-0.079955	1.141216	0.126134	-3.577211e-01
8	0.989572	1.003363	0.000000	0.000000	1.003444	0.009594	9.344107e-07
9	0.989572	1.003363	0.000000	0.000000	1.003444	0.009594	9.344108e-07
10	0.807349	0.757135	-0.453031	-0.218289	1.114265	0.271901	5.203561e-02
11	0.866743	0.791943	-0.230422	-0.075699	1.091069	0.215261	1.064363e-02
12	1.073113	0.973013	0.000000	0.000000	0.916460	0.009097	9.765388e-07
13	1.073113	0.973013	0.000000	0.000000	0.916460	0.009097	9.765389e-07
14	0.967562	1.044105	0.003387	0.218926	0.660610	0.114103	8.478428e-02
15	0.823556	1.007926	-0.049645	0.195954	0.689651	0.091325	6.914803e-02
16	0.928942	1.055156	0.000000	0.000000	0.939434	0.010325	1.084500e-06
17	0.928942	1.055157	0.000000	0.000000	0.939434	0.010325	1.084500e-06
18	0.771169	1.032614	0.014816	0.049583	1.095744	0.151763	-2.528617e-01
19	0.924726	0.924052	-0.088329	-0.315591	1.120163	0.205937	-2.005126e-02

	Q21	Q22	R11	R12	R21	R22
0	0.000001	0.010878	0.102497	0.010768	0.009182	0.096302
1	0.000001	0.010878	0.102497	0.010768	0.009182	0.096302
2	-0.246361	0.154221	0.158285	0.002389	-0.058235	0.107512
3	-0.181571	0.189579	0.196283	0.117786	-0.020030	0.122268
4	0.000001	0.009380	0.109075	0.009379	0.010691	0.093214
5	0.000001	0.009380	0.109075	0.009379	0.010691	0.093214
6	0.172534	0.210728	0.264564	-0.084258	0.046427	0.182231
7	0.339459	0.189625	0.132048	-0.028706	-0.028922	0.364598
8	0.000001	0.010617	0.100243	0.010192	0.010793	0.098955
9	0.000001	0.010617	0.100243	0.010192	0.010793	0.098955
10	-0.226278	0.127284	0.093353	0.094953	-0.127200	0.135347
11	-0.048214	0.103792	0.080171	0.073448	-0.072587	0.150975
12	0.000001	0.009699	0.101522	0.009060	0.010467	0.095675
13	0.000001	0.009699	0.101522	0.009060	0.010467	0.095675
14	-0.153745	0.071534	0.164993	0.135771	-0.179268	0.083478
15	0.025487	0.210161	0.099601	0.187232	-0.204456	0.100249
16	0.000001	0.010019	0.108067	0.010244	0.010489	0.096365
17	0.000001	0.010019	0.108067	0.010244	0.010489	0.096365

18 0.219972 0.226074 0.134067 -0.157172 0.226739 0.141666
 19 -0.005987 0.158983 0.155423 -0.006384 0.113550 0.181703

3.1 Particle Filter (pfilter)

[outline: 1. purpose/role 2. implementation details in pypomp 3. outputs/results 4. remarks/highlights]

The particle filter algorithm (sequential Monte Carlo, SMC) (Arulampalam et al. (2002), King, Nguyen, and Ionides (2016)) is the core classical likelihood evaluation tool in POMP model inference. It uses Monte Carlo techniques to sequentially estimate the integrals in the prediction and filtering recursions, and produce an unbiased Monte Carlo estimate of the likelihood of the observed data under a parameter set. The likelihood estimate is essential for other parameter inference algorithms such as iterative filtering. In **pypomp**, the basic particle filter is implemented under **pfilter()**. The algorithm is summarized as Algorithm 1,

Algorithm 1 Sequential Monte Carlo (SMC, or particle filter) in `textttpypomp`: `LG_obj.pfilter(J=J, reps=reps, key=key)`, where `LG_obj` is a class `Pomp` object with components `rinit`, `rproc`, `dmeas`, `rmeas`, `ys`, and `theta`

Require Simulator for $\text{process}_n(\cdot | x_n; \theta)$; Evaluator for $f_{Y_n | X_n}(y_n | x_n; \theta)$; Simulator for $f_{X_0}(x_0; \theta)$; Parameter θ ; Data $y_{1:N}^*$; Number of particles J .

Initialize particles: $X_{0,j}^{F,\theta} \sim f_{X_0}(\cdot; \theta)$ for $j = 1:J$.

For $n = 1, \dots, N$:

Simulate prediction particles: $X_{n,j}^{P,\theta} \sim \text{process}_n(\cdot | x_n; \theta)$ for $j = 1:J$.

Evaluate weights and measurement density: $w(n, j) = f_{Y_n | X_n}(y_n^* | X_{n,j}^{P,\theta}; \theta)$ for $j = 1:J$.

Normalize weights $\tilde{w}(n, j) = \frac{w(n, j)}{\sum_{m=1}^J w(n, m)}$ for $j = 1:J$.

Select resample indices $k_{1:J}$ with $P(k_j = m) = \tilde{w}(n, m)$.

Obtain resampled particles $X_{n,j}^{F,\theta} = X_{n,k_j}^{P,\theta}$ for $j = 1:J$.

Compute conditional log likelihood:

$$\hat{\ell}_{n|1:n-1} = \log \left(\frac{1}{J} \sum_{m=1}^J w(n, m) \right).$$

Return Log likelihood estimate $\hat{\ell}(\theta) = \sum_{n=1}^N \hat{\ell}_{n|1:n-1}$; filter samples $X_{n,1:J}^{F,\theta}$ for $n = 1:N$.

Complexity: $\mathcal{O}(J)$

In **pypomp**, the **pfilter()** functions is internally run in a JIT-compiled kernel `_vmapped_pfilter_internal2`, which applies `jax.vmap` over replicates and sharded parameters on `theta`. This enables the parallel execution on CPU/GPU and ensures the computational efficiency. In implementation,

the weights and likelihood are stored in a log scale, preventing underflow. There are optional arguments (`CLL`, `ESS`, `filter_mean`, `prediction_mean`) control wheter additional diagnostic arrays are allocated and updated. The algorithm works for flexible and extendable models through the user-specified `Pomp` class components (`rinit`, `rproc`, `dmeas`, `reams`), number of particles J as well as other parameters. Each call generates a `PompPFilterResult` object, which is then appended to `object.results_history` attribute, containing the execution time, evaluated log-likelihoods, algorithm parameters used, as well as model diagnostic elements at each time included if their respective boolean flags are set to `True`. Here `PompPFilterResults` class is derived from `PompBaseResult` and provides standardized storage and methods for `pfilter()` results. This return design supports multi-stage inference workflow and ensures reproducibility. For example, suppose we run

```
LG_obj_2 = pp.Pomp(
    rinit=rinit,
    rproc=rproc,
    dmeas=dmeas,
    rmeas=rmeas,
    t0=0.0,
    nstep=1,
    ys=ys,
    theta=theta_list,
    statenames=["X1", "X2"],
)

LG_obj_2.pfilter(J=1000, reps=10, key=subkey)

LG_obj_2.pfilter(
    J=1000,
    reps=10,
    key=subkey,
    CLL=True,
    ESS=True,
    filter_mean=True,
    prediction_mean=True,
)
```

where J is the number of particles used and *reps* is the number of particle filtering replicates to run for each parameter set provided in the `Pomp` object or as an optional argument to `pfilter()`. Because `LG_obj2.results_history` begins as an empty list here when the model is constructed, the `PompPFilterResult` object results are appended at `LG_obj_2.results_history[0]` and `LG_obj_2.results_history[1]` respectively. Both of these two objects contain the following attributes which are not `None`:

- **method**: The method that was run. In this case, "pfilter".
- **execution time**: The execution time if `track_time = True`
- **key**: The RNG key used.
- **theta**: The list of parameter sets used.
- **logLiks**: An `xarray.DataArray` of particle filter log-likelihood estimates. Evaluated log-likelihoods returned from the internal kernel are reshaped into `(n_theta, reps)` based on the number of parameter sets and replicates.
- **J**: The number of particles used.
- **reps**: The number of replicates.
- **thresh**: Threshold value for resampling used, which is 0 by default.

Meanwhile, `LG_obj_2.results_history[1]` also contains the following items that are not contained in `LG_obj_2.results_history[0]` :

- **CLL**: An `xarray.DataArray` containing the conditional loglikelihood over observed time steps (`theta, replicate, time`).
- **ESS**: An `xarray.DataArray` containing the effective sample size over observed time steps (`theta, replicate, time`).
- **filter_mean**: An `xarray.DataArray` containing the filtered mean over observed time steps (`theta, replicate, time, state`).
- **prediction_mean**: An `xarray.DataArray` containing the prediction mean over observed time steps. (`theta, replicate, time, state`).

The result `PompPFilterResult` also provides the following functions for further analysis: - `to_dataframe()` provides the summary data frame of per-parameter-set log-likelihood estimates via log-mean-exp trick with Monte Carlo standard error. - `traces()` returns traces data frame with one row per replicate. - `print_summary` prints brief readable summary for the results with top 5 evaluated log-likelihood results.

A key design is that all runs across multiple parameter sets and replications are stored in one `results_history` list under the corresponding `Pomp` object. This avoids object duplication, and increases reproducibility by facilitating pickling `Pomp` objects obtained from many different search strategies, loading them into a new Python session, and then comparing the results to see how they vary with different algorithmic parameters.

3.2 Differentiated Measurement Off-Parameter (DMOP) particle filter

([TO BE REWRITTEN, FOLLOWING ISSUE 2])

[\[Motivation for DMOP\]](#) Classical particle filters for POMP models produce unbiased Monte Carlo estimates, but are non-differentiable in model parameters θ , due to the discrete random sampling of particles. This prevents the wide use of AD and gradient-based optimization in POMP dynamical systems. The differentiable measurement off-parameter particle filter (DMOP- α , Tan, Hooker, and Ionides (2024)) provides a solution: it treats a *differentiable*

likelihood estimator produced by a variant of the standard particle filter as an objective and apply AD to obtain $\nabla_{\theta} \hat{\ell}(\theta)$ for likelihood-based optimal parameter search. The derivative estimate from DMOP- α is essential for further stochastic gradient inference methods, such as iterated filtering with automatic differentiation (IFAD), where it employs stochastic gradient descent using DMOP- α . A tuning discount factor $\alpha \in [0, 1]$ controls the bias-variance trade-off of the gradient estimator: larger α may result in lower bias and higher variance, while smaller α can reduce the variance with the cost of increasing bias. In implementation, DMOP- α does not require heavy diagnostics or multiple replicates, and doesn't require time tracking function. It only needs a stable, differentiable of the (log-) likelihood to feed into `jax.grad` and `jax.value_and_grad`. The pseudocode is shown in 2

Algorithm 2 DMOP- α in pypomp: `jax.value_and_grad(LG_obj.mop(J=J, reps=reps, key=key, alpha=alpha))`. `LG_obj` is a `Pomp` object with components `rinit`, `rproc`, `dmeas`, `ys`, `theta`.

Require Simulator $\text{process}_n(\cdot|x_n; \theta)$; Evaluator for $f_{Y_n|X_n}(y_n^*|x_n, \theta)$; Simulator for $f_{X_0}(\cdot; \theta)$; Evaluation parameter θ ; Data $y_{1:N}^*$; Number of particles J ; Discount factor α .

Initialize particles $X_{0,j}^{F,\theta} \sim f_{X_0}(\cdot; \theta)$, weights $w_{0,j}^{F,\theta} = 1$ for $j = 1 : J$.

For $n = 1, \dots, N$:

 Simulate prediction particles, $X_{n,j}^{P,\theta} \sim \text{process}_n(\cdot | X_{n-1,j}^{F,\theta}; \theta)$ for $j = 1 : J$.

 Accumulate discounted prediction weights, $w_{n,j}^{P,\theta} = (w_{n-1,j}^{F,\theta})^\alpha$.

 Evaluate measurement density, $g_{n,j}^\theta = f_{Y_n|X_n}(y_n^* | X_{n,j}^{P,\theta}; \theta)$ for $j = 1 : J$.

 Compute conditional likelihood in version B: $L_n^{B,\theta,\alpha} = \sum_{j=1}^J g_{n,j}^\theta w_{n,j}^{P,\theta} / \sum_{j=1}^J w_{n,j}^{P,\theta}$.

 Select resampling indices $k_{1:J}$ with $P(k_j = m) \propto g_{n,m}^\theta$.

 Obtain resampled particles $X_{n,j}^{F,\theta} = X_{n,k_j}^{P,\theta}$.

 Calculate corrected weights $w_{n,j}^{F,\theta} = w_{n,k_j}^{P,\theta} g_{n,k_j}^\theta / \text{stop_gradient}(g_{n,k_j}^\theta)$.

 Compute conditional likelihood in version A: $L_n^{A,\theta,\alpha} = L_n^\phi \cdot \sum_{j=1}^J w_{n,j}^{F,\theta} / \sum_{j=1}^J w_{n,j}^{P,\theta}$.

Forward Pass: AD constructs a computation graph to evaluate the likelihood estimate $\hat{L}^A(\theta) = \prod_{n=1}^N L_n^{A,\theta,\alpha}$ or $\hat{L}^B(\theta) = \prod_{n=1}^N L_n^{B,\theta,\alpha}$.

Backward Pass: AD propagates gradients through the computation graph and evaluates gradient estimate $\hat{\mathcal{D}}^A(\theta)$ or $\hat{\mathcal{D}}^B(\theta)$.

Return: likelihood estimate $\hat{L}^A(\theta)$ or $\hat{L}^B(\theta)$, and gradient estimate $\hat{\mathcal{D}}^A(\theta)$ or $\hat{\mathcal{D}}^B(\theta)$.

The key building block of the DMOP- α is the MOP- α algorithm, a variant of the standard particle filter designed to make the discontinuous Monte Carlo resampling process independent of the target parameter θ . It evaluates the likelihood at the evaluation parameter θ but draws resampling indices using weights computed at another baseline behavior parameter ϕ . This renders the likelihood estimates differentiable w.r.t. θ , further enabling AD.

In DMOP- α , the same idea is implemented equivalently by setting $\phi \equiv \text{stop_gradient}(\theta)$ in JAX. This prevent its input from being differentiated. The filtering process therefore runs

$\theta = \phi$, assuming the evaluation parameter and the behavior parameters have identical values. This avoids the need to explicitly introduce new parameters while preserving the differentiable property. Consequently, the likelihood evaluation of DMOP is identical to that of the standard particle filter, and it does not require implement a fully separate MOP- α procedure. This design simplifies the algorithm and still ensures the valid likelihood and gradient estimator, while maintaining differentiability with respect to θ . That is, DMOP computes the derivative of the likelihood estimate obtained by MOP when $\theta = \phi$. The pseudocode of MOP is shown in 3.

Algorithm 3 MOP(α), Measurement off-policy sequential Monte Carlo

Require Simulator for process $_n(\cdot|x_n;\theta)$; Evaluator for $f_{Y_n|X_n}(y_n | x_n;\theta)$; Simulator for $f_{X_0}(x_0;\theta)$; Evaluation Parameter θ ; Behavior Parameter ϕ ; Data $y_{1:N}^*$; Number of particles J .

Initialize filter particles $X_{0,j}^{F,\theta} \sim f_{X_0}(\cdot;\theta)$ and $w_{0,j}^{F,\theta} = 1$ for $j = 1:J$.

For $n = 1, \dots, N$:

Simulate prediction particles: $X_{n,j}^{P,\theta} \sim f_{X_n|X_{n-1}}(\cdot | X_{n-1,j}^{F,\theta}; \theta)$ for $j = 1:J$.

Accumulate discounted Prediction weight: $w_{n,j}^{P,\theta} = (w_{n-1,j}^{F,\theta})^\alpha$ for $j = 1:J$.

Evaluate measurement density: $g_{n,j}^\theta = f_{Y_n|X_n}(y_n^* | X_{n,j}^{P,\theta}; \theta)$ for $j = 1:J$.

Compute conditional likelihood under θ in version B: $L_n^{B,\theta,\alpha} = \sum_{j=1}^J g_{n,j}^\theta w_{n,j}^{P,\theta} / \sum_{j=1}^J w_{n,j}^{P,\theta}$.

Compute conditional likelihood under ϕ : $L_n^\phi = \frac{1}{J} \sum_{m=1}^J g_{n,m}^\phi$.

Select resample indices $k_{1:J}$ with $P(k_j = m) \propto \tilde{g}_{n,m}^\phi$.

Obtain resampled particles $X_{n,j}^{F,\theta} = X_{n,k_j}^{P,\theta}$ for $j = 1:J$.

Calculate resampled corrected weights: $w_{n,j}^{F,\theta} = w_{n,j}^{P,\theta} \times \frac{g_{n,j}^\theta}{g_{n,j}^\phi}$ for $j = 1:J$.

Compute conditional likelihood under θ in version A: $L_n^{A,\theta,\alpha} = L_n^\phi \cdot \sum_{j=1}^J w_{n,j}^{F,\theta} / \sum_{j=1}^J w_{n,j}^{P,\theta}$.

Return: Likelihood estimate: $\hat{L}(\theta) = \prod_{n=1}^N L_n^{A,\theta,\alpha}$ or $\hat{L}(\theta) = \prod_{n=1}^N L_n^{B,\theta,\alpha}$.

In `pypomp`, DMOP- α is implemented by wrapping the JAX-compiled `mop()` function inside JAX AD. It is a `Pomp` class method calling an internal `_mop_internal()`, which uses static arguments and `jax.lax.fori_loop` for efficiency, and stores weights and likelihoods in log scale to prevent underflow. Systematic sampling is involved for the resampling process. Users can directly apply `jax.grad` and `jax.value_and_grad` to `mop()` to obtain gradients.

In practice, the `mop()` provides an auto-differentiable alternative to `pfilter()`, as a lightweight version. Unlike `pfilter()`, which provides the choice of multiple replicates and diagnostic summaries, `mop()` in `pypomp` focuses purely on producing differentiable likelihood estimates. This makes the algorithm suitable for AD and stochastic gradient descent algorithms in accelerated iterated filtering algorithms by setting the function as input of `jax.value_and_grad()` and `jax.grad()`. Beyond a well-constructed `Pomp` object, the method requires only the number of particles J , the evaluation parameter `theta`, a PRNG key, and the cooling factor α . Results are stored in the `results_history` attribute as a list of `jax.Array` objects, one for each pa-

parameter set, representing the estimated log-likelihood of the observations. For example, we run

[AA: Update for removal of MOP method.]

```
LG_obj_3 = pp.Pomp(
    rinit=rinit,
    rproc=rproc,
    dmeas=dmeas,
    rmeas=rmeas,
    t0=0.0,
    nstep=1,
    ys=ys,
    theta=theta_list,
    statenames=["X1", "X2"],
)

# LG_obj_3.mop(J=1000, key=subkey, alpha=0.97)
```

Generally, the `result_history` only updates:

- log-likelihood(s): a list of `jax.Array` containing the estimated log-likelihood(s) of the observed data given the model parameters. Always a list, even if only one theta is provided.

This design highlights `mop()` as a lightweight but essential building block for accelerated gradient-based inference methods within `pypomp`. By combining `jax.grad` and `jax.value_and_grad` around `mop()`, the package effectively implements the DMOP algorithm, enabling stochastic-gradient updates for likelihood-based inference on POMP models.

3.3 Iterated Filtering

Iterated filtering (IF2) provides a Monte Carlo approach to estimate parameters by combining particle filtering with sequential random-walk perturbations and a cooling schedule to approximate maximum likelihood estimates when the likelihood is analytically intractable but can be unbiasedly estimated through sequential Monte Carlo methods. Compared with a plain particle filter or MOP method, IF2 not only evaluates the likelihoods but also systematically updates parameters across iterations. IF2 enables systematic exploration of the parameter space in early iterations via perturbation and gradual stabilization around high-likelihood regions when the perturbation variance decreases. The IF2 algorithm contains M iterations, each using J particles. At iteration m , parameters are perturbed by a Gaussian random

walk with standard deviations scaled by a cooling factor. At each observation time, the likelihood evaluations and resampling are performed and parameter estimates are obtained. The perturbation magnitudes are progressively reduced across iterations, resulting in convergence to a Monte Carlo approximation of the maximum likelihood estimate, which is an optimal parameter estimate.

Algorithm 4 Iterated Filtering (IF2)

Require: Starting parameter θ_0 ; simulator for $f_{X_0}(x_0; \theta)$; simulator for $f_{X_n|X_{n-1}}(x_n | x_{n-1}; \theta)$; evaluator for $f_{Y_n|X_n}(y_n | x_n; \theta)$; data $y_{1:N}$; labels $I \subset \{1, \dots, p\}$ for IVPs; fixed lag L ; number of particles J ; number of iterations M ; cooling rate a , $0 < a < 1$; perturbation scales $\sigma_{1:p}$; initial scale multiplier $C > 0$.

- 1: **for** $m = 1$ to M **do**
- 2: Initialize parameters: $\Theta_{0,j,i}^P \sim \text{Normal}([\theta_{m-1}]_i, (Ca^m \sigma_i)^2)$ for $i \in 1:p$, $j \in 1:J$.
- 3: Initialize states: simulate $X_{0,j}^F \sim f_{X_0}(\cdot; \Theta_{0,j}^P)$ for $j = 1:J$.
- 4: Initialize filter mean: $\bar{\theta}_0 = \theta_{m-1}$.
- 5: Define $[V]_i = (C^2 + 1)a^{2m}\sigma_i^2$.
- 6: **for** $n = 1$ to N **do**
- 7: Perturb parameters: $\Theta_{n,j,i}^P \sim \text{Normal}([\Theta_{n-1,j}^F]_i, (a^m \sigma_i)^2)$ for $i \notin I$, $j = 1:J$.
- 8: Simulate prediction particles: $X_{n,j}^P \sim f_{X_n|X_{n-1}}(\cdot | X_{n-1,j}^F; \Theta_{n,j}^P)$ for $j = 1:J$.
- 9: Evaluate weights: $w(n, j) = f_{Y_n|X_n}(y_n^* | X_{n,j}^P; \Theta_{n,j}^P)$ for $j = 1:J$.
- 10: Normalize weights: $\tilde{w}(n, j) = \frac{w(n, j)}{\sum_{u=1}^J w(n, u)}$.
- 11: Resample indices $k_{1:J}$ with $\Pr[k_u = j] = \tilde{w}(n, j)$.
- 12: Resample particles: $X_{n,j}^F = X_{n,k_j}^P$ and $\Theta_{n,j}^F = \Theta_{n,k_j}^P$ for $j = 1:J$.
- 13: Filter mean: $[\bar{\theta}_n]_i = \sum_{j=1}^J \tilde{w}(n, j)[\Theta_{n,j}^P]_i$ for $i \notin I$.
- 14: Prediction variance: $[V_{n+1}]_i = (a^m \sigma_i)^2 + \sum_{j=1}^J \tilde{w}(n, j) ([\Theta_{n,j}^P]_i - [\bar{\theta}_n]_i)^2$ for $i \notin I$.
- 15: **end for**
- 16: Update non-IVPs: $[\theta_m]_i = [\theta_{m-1}]_i + [V]_i \sum_{n=1}^N ([\bar{\theta}_n]_i - [\theta_{m-1}]_i)$ for $i \notin I$.
- 17: Update IVPs: $[\theta_m]_i = \frac{1}{J} \sum_{j=1}^J [\Theta_{L,j}^F]_i$ for $i \in I$.
- 18: **end for**

Ensure: Monte Carlo maximum likelihood estimate θ_M .

In `pypomp`, the IF2 algorithm is implemented in the method `mif()`. It follows the structure of the IF2 implementation in the R package `pomp`, while introducing JAX-based vectorization and compilation for improved computational performance.

The argument `rw_sd` specifies the random walk standard deviations for parameter perturbations. The perturbation scales are encapsulated in a `RWSigma` object, which is passed to the method through the `rw_sd`. The `RWSigma` object stores a mapping from parameter names to non-negative perturbation values and distinguishes between initial-value parameters (IVPs)

and non-initial parameters, allowing different perturbation behavior at the initial time point and at subsequent time steps. The float-type cooling rate `a` specifies the decay of perturbation magnitudes across iterations. Together, `rw_sd` and `a` jointly control the trade-off between global exploration and local convergence of the parameter estimates.

The `mif()` updates the internal state of the `Pomp` object directly. In particular, `LG_obj.theta` is replaced with the parameter estimate from the end of the last iteration. The traces of log-likelihoods and parameter estimates are stored in `LG_obj.results_history` as a `xr.DataArray` object. This array has dimensions (`replicate`, `iteration`, `variable`), where the `variable` dimension includes the log-likelihood estimations and all POMP model parameters. For reproducibility, each call updates and stores a child key into `LG_obj.fresh_key`, allowing subsequent method calls to safely omit an explicit assignment on `key` argument.

Each call generates a `PompMIFResult` object, another dataclass derived from `PompBaseResult`. The object is appended to `object.results_history` attribute, containing the IF2 trace outputs, algorithm settings and execution context:

- `method`: The method that was run. In this case, "mif".
- `execution time`: The execution time if `track_time = True`
- `key`: The RNG key used.
- `theta`: The list of parameter sets used.
- `traces_da`: An `xarray.DataArray` of IF2 trace outputs. The array is indexed by (`replicate`, `iteration`, `variable`), where `variable` includes the estimated loglikelihood and all model parameters.
- `J`: The number of particles used.
- `M`: The number of IF2 iterations.
- `rw_sd`: an `RWSigma` object representing the random walk perturbation specification.
- `a`: Cooling fraction.
- `thresh`: Threshold value for resampling used, which is 0 by default.

Similar to `PompPFilterResult` class, `PompMIFResult` object contains the `to_dataframe()`, `traces()`, and `print_summary()` as helper functions

```
key, subkey = jax.random.split(key)
```

```
LG_obj_4 = pp.Pomp(
    rinit=rinit,
    rproc=rproc,
    dmeas=dmeas,
    rmeas=rmeas,
    t0=0.0,
    nstep=1,
    ys=ys,
    theta=theta_list,
```

```

    statenames=["X1", "X2"],
)

rw_sd = pp.RWSigma(
  sigmas={
    "A11": 0.02,
    "A12": 0.02,
    "A21": 0.02,
    "A22": 0.02,
    "C11": 0.02,
    "C12": 0.02,
    "C21": 0.02,
    "C22": 0.02,
    "Q11": 0.02,
    "Q12": 0.02,
    "Q21": 0.02,
    "Q22": 0.02,
    "R11": 0.02,
    "R12": 0.02,
    "R21": 0.02,
    "R22": 0.0,
  },
  init_names=[],
)

LG_obj_4.mif(rw_sd=rw_sd, J=1000, M=2, a=0.5, key=subkey)

```

The `pyrmp` implementation of IF algorithm in `mif()` inherits the structure of R's `pomp` package while adding JAX compilation, providing a substantial performance improvements over R implementation.

3.4 Iterated Filtering with Automatic Differentiation

Iterated filtering with automatic differentiation (IFAD) (Tan, Hooker, and Ionides 2024) is a two-stage method that combines the complementary advantages of iterated filtering (IF2) and a new proposed MOP- α gradient method. IF2 is effective at global exploration of the parameter space and can quickly approach the neighborhood of the global maximum, but it struggles to achieve local convergence to the global maximum once it has reached a high-likelihood neighborhood. In contrast, by using the differentiable particle filter, the MOP- α gradient method can conduct local refinements via gradients (and optionally Hessians) in the parameter space, but shows weaker ability at conducting a global exploration (Tan, Hooker, and Ionides

2024). IFAD addresses this tradeoff by first running IF2 to approach the promising region and then switching to a gradient-based method local search to further refine the parameter estimates.

The Algorithm 5 outlines the IFAD workflow:

Algorithm 5 IFAD: Iterated Filtering with Automatic Differentiation

Require: Number of particles J , timesteps N , IF2 cooling schedule η_m , MOP- α discounting parameter α , initial parameter θ_0 , iteration index $m = 0$.

- 1: Run IF2 until initial “convergence” under cooling schedule η_m , or for a fixed number of iterations, to obtain $\{\Theta_j, j = 1, \dots, J\}$.
- 2: Set $\theta_m := \frac{1}{J} \sum_{j=1}^J \Theta_j$.
- 3: **while** procedure not converged **do**
- 4: Run Algorithm 3 (MOP- α filter) to obtain $\hat{\ell}(\theta_m)$.
- 5: Obtain gradient and Hessian:

$$g(\theta_m) = \nabla_{\theta_m} (-\hat{\ell}(\theta_m)), \quad H(\theta_m) \quad \text{s.t. } \lambda_{\min}(H(\theta_m)) > c.$$

- 6: Update parameter:

$$\theta_{m+1} := \theta_m - \eta_m H(\theta_m)^{-1} g(\theta_m).$$

- 7: Set $m := m + 1$.

- 8: **end while**

Ensure: Return $\hat{\theta} := \theta_m$.

In `pypomp`, the MOP- α gradient local search stage is implemented by the `train()`, which utilizes JAX’s automatic differentiation features to iteratively optimize MOP- α objective. While IFAD is not implemented as an independent function, users can always run `mif()` and `train()` to execute the two phases sequentially. For example, the following code first runs IF2 for M_1 iterations to obtain a high-likelihood starting point, and then applies Newton-type optimization for M_2 iterations to refine the estimate:

```
key, subkey = jax.random.split(key)
LG_obj_5 = pp.Pomp(
    rinit=rinit,
    rproc=rproc,
    dmeas=dmeas,
    rmeas=rmeas,
    t0=0.0,
    nstep=1,
    ys=ys,
    theta=theta_list,
    statenames=["X1", "X2"],
```

```

)
LG_obj_5.mif(rw_sd=rw_sd, J=1000, M=3, a=0.5, key=subkey)
eta = {name: 0.0025 for name in LG_obj_5.canonical_param_names}
LG_obj_5.train(
    J=1000,
    M=3,
    eta=eta, # learning rate
    optimizer="Adam",
)

```

In `pypomp`, `train()` serves as a gradient-based optimizer for exploring the parameter space. Each iteration follows the standard optimization working flow: starting from the current parameter estimate, it computes gradient (and Hessian) of the log-likelihood via `jax.grad()` and `jax.Hessian()`, then utilizes the choosen internal optimizer to update the direction, obtains the learning rate, and finally updates the parameters as well as evaluates the new likelihood under the updated parameters. `train()` supports several common gradient-based optimizers for exploring the parameter space, including Newton’s method, weighted Newton’s method, Broyden-Fletcher-Goldfarb-Shanno (BFGS) algorithm, and stochastic gradient descent method (SGD). In addition, An optional Armijo backtracking line search method (Wills and Schön 2021) is available for adaptively determining the learning rate (step size) under stochastic Quasi-Newton methods.

In `train()`, there is a key argument `n_monitors`, controlling the number of log-likelihood values are evaluated during each training step. When setting `n_monitors=0`, the algorithm totally skip the evaluating process and conduct the gradient-only optimization. It is the fastest choice, but doesn’t support the line search method for learning rate determination, as it utilizes the estimated log-likelihood as the current objective function value. `n_monitors=1` is a lightest monitoring form, where users can get a one log-likelihood evaluation per training step. Larger `n_monitors` reduces the log-likelihood estimate variance in particle filtering using Monte Carlo average at additional computational cost. There is another useful Boolean flag, `scale`, which controls whether the search direction is normalized to unit length. A normalized search direction can provide a more conservative update when the gradients or Hessians are unstable.

Similarly to `mif()`, each call to `train()` creates a `PompTrainResult` object, which contains the algorithmic settings and an `xarray.DataArray` trace `traces_da` with dimensions (`replicate`, `iteration`, `variable`). The result object is appended into `results_history`.

The core highlight of IFAD in algorithmic implementation lies in its internal automatic differentiation and just-in-time compilation powered by JAX. Via JAX, gradients and Hessian matrices for the MOP- α log-likelihood function can be directly obtained without manual derivation, ensuring correctness, efficiency and operability. This also enables flexible use of advanced

gradient-based optimizers in each training step, such as Newton’s method, quasi-Newton methods, BFGS, gradient descent and stochastic line search. Additionally, JAX’s GPU compilation capabilities enable these optimization processes to execute efficiently, making IFAD not only theoretically feasible but also capable of effectively handling large complex POMP models in practical algorithmic implementations.

4 Data Analysis with pypomp / Tutorials and data analysis examples

We first show the benchmarking results of our software running on the dacca dataset.

Table: Main arguments to the `Pomp` class and related constructor objects.

For `mif2`, `train` and `ifad`, we started with a param box including the true (default) parameters, 0.5 x true parameters and 1.5 x true parameters. Each replicate, we chose the best loglikelihood estimate among the three starting points. In this table, the `pfilter` and `mop` are evaluated at the true ()

Software	Backend	Run Level	Method	Evaluated Loglikli-hood (logexp-mean)	SE	Average Run Time(s)	Maximum Memory Usage (MiB)
pypomp	CPU	3	pfilter	-3748.41	0.43	2.23	739
pypomp	CPU	3	mop	-3748.02	0.32	3.31	713
pypomp	CPU	3	IF2	-3770.37	1.00	639.52	834
pypomp	CPU	3	pure train()				
pypomp	CPU	3	IFAD				

```

pypomp | GPU | 3 | pfilter | -3747.58 | 0.35 | 0.26 | 947 |
pypomp | GPU | 3 | mop | -3747.61 | 0.71 | 0.39 | 745 |
pypomp | GPU | 3 | IF2 | -3763.69 | 0.42 | 28.90 | 1638.4 |
pypomp | GPU | 3 | pure train() | -20000 | | |
pypomp | GPU | 3 | IFAD | | | |

pypomp | CPU | 4 | pfilter | -3748.16 | 0.13 | 7.32 | 852 |
pypomp | CPU | 4 | mop | -3742.11 | 0.15 | 14.53 | 790 |
pypomp | CPU | 4 | mif2 | -3752.48 | 0.27 | 198.41 | 1126.4
pypomp | GPU | 4 | pfilter | -3748.15 | 0.18 | 0.41 | 857 |

```



```
pypomp | GPU | 4 | mop | -3748.34 | 0.12 | 0.49 | 821 |
pypomp | GPU | 4 | mif2 | -3752.48 | 0.27 | 198.41 | 2048 |
```

Here the optimizer for `train()` and `ifad()` is set to the Adam optimizer. The maximum memory usage is obtained from Greatlakes summary. From the table, we can see three points: 1. **Statistical Correctness:** The log-likelihood estimates from `pfilter()` and `mop()` are close to each other, with `mop()` showing slightly higher values, which is expected as `mop()` is designed to provide a differentiable likelihood estimate that can be optimized. 2. **IF2 Performance:** The IF2 method shows a lower log-likelihood estimate compared to `pfilter()` and `mop()`, which may be due to the fact that IF2 is an iterative method that may not have fully converged within the given number of iterations, or it may be more sensitive to the choice of starting parameters and cooling schedule. But overall, the IF2 captures the overall likelihood structure and provides a reasonable estimate, albeit with higher variance. 3. **GPU Acceleration:** The GPU implementations of `pfilter()` and `mop()` show significantly reduced run (give the CPU/GPU run time ratio). 4. **Method Comparison:** (Dmop, IFAD, IF2...)

We also use the R package `pomp` to run the same methods as the baseline and compare the results with `pypomp`.

This section demonstrates:

- Log-likelihood profiling
- GPU benchmarking
- Conditional log-likelihood residuals # Discussion {#sec-discussion}

- Abkemeier, Aaron, Jun Chen, Edward Ionides, Jesse Wheeler, and Kevin Tan. 2024. “Py-pomp.”
- Arulampalam, M. S., S. Maskell, N. Gordon, and T. Clapp. 2002. “A Tutorial on Particle Filters for Online Nonlinear/Non-Gaussian Bayesian Tracking.” *IEEE Transactions on Signal Processing* 50 (2): 174–88. <https://doi.org/10.1109/78.978374>.
- Asfaw, Kidus, Joonha Park, Aaron A. King, and Edward L. Ionides. 2024. “spatPomp: An R Package for Spatiotemporal Partially Observed Markov Process Models.” *Journal of Open Source Software* 9 (104): 7008. <https://doi.org/10.21105/joss.07008>.
- Auger-Méthé, Marie, Ken Newman, Diana Cole, Fanny Empacher, Rowenna Gryba, Aaron A. King, Vianey Leos-Barajas, et al. 2021. “A Guide to State-Space Modeling of Ecological Time Series.” *Ecological Monographs* 91 (4): e01470. <https://doi.org/10.1002/ecm.1470>.
- Bjørnstad, Ottar N., and Bryan T. Grenfell. 2001. “Noisy Clockwork: Time Series Analysis of Population Fluctuations in Animals.” *Science* 293: 638–43. <https://doi.org/10.1126/science.1062226>.
- Blackwood, J. C., D. G. Streicker, S. Altizer, and P. Rohani. 2013. “Resolving the Roles of Immunity, Pathogenesis, and Immigration for Rabies Persistence in Vampire Bats.” *Proceedings of the National Academy of Sciences of the United States of America* 110 (51): 20837–42. <https://doi.org/10.1073/pnas.1308817110>.

- Bradbury, James, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Neca, et al. 2018. “JAX: Composable Transformations of Python+NumPy Programs.”
- Bretó, Carles. 2014. “On Idiosyncratic Stochasticity of Financial Leverage Effects.” *Statistics & Probability Letters* 91: 20–26. <https://doi.org/10.1016/j.spl.2014.04.003>.
- Bretó, Carles, Daihai He, Edward L. Ionides, and Aaron A. King. 2009. “Time Series Analysis via Mechanistic Models.” *The Annals of Applied Statistics* 3 (1): 319–48. <https://doi.org/10.1214/08-AOAS201>.
- Bretó, Carles, Jesse Wheeler, Aaron A. King, and Edward L. Ionides. 2025. “panelPomp: Analysis of Panel Data via Partially Observed Markov Processes in R.” arXiv. <https://doi.org/10.48550/arXiv.2410.07934>.
- Fox, S. J., M. Lachmann, M. Tec, R. Pasco, S. Woody, Z. Du, X. Wang, et al. 2022. “Real-Time Pandemic Surveillance Using Hospital Admissions and Mobility Data.” *Proceedings of the National Academy of Sciences* 119 (7): e2111870119. <https://doi.org/10.1073/pnas.2111870119>.
- He, Daihai, Edward L. Ionides, and Aaron A. King. 2010. “Plug-and-Play Inference for Disease Dynamics: Measles in Large and Small Populations as a Case Study.” *Journal of The Royal Society Interface* 7 (43): 271–83. <https://doi.org/10.1098/rsif.2009.0151>.
- Ionides, E. L., C. Breto, and A. A. King. 2006. “Inference for Nonlinear Dynamical Systems.” *Proceedings of the National Academy of Sciences* 103 (49): 18438–43. <https://doi.org/10.1073/pnas.0603181103>.
- Ionides, E. L., C. Breto, J. Park, R. A. Smith, and A. A. King. 2017. “Monte Carlo Profile Confidence Intervals for Dynamic Systems.” *Journal of The Royal Society Interface* 14 (132): 20170126. <https://doi.org/10.1098/rsif.2017.0126>.
- Ionides, Edward L., Dao Nguyen, Yves Atchadé, Stilian Stoev, and Aaron A. King. 2015. “Inference for Dynamic and Latent Variable Models via Iterated, Perturbed Bayes Maps.” *Proceedings of the National Academy of Sciences* 112 (3): 719–24. <https://doi.org/10.1073/pnas.1410597112>.
- King, Aaron A., Edward L. Ionides, Mercedes Pascual, and Menno J. Bouma. 2008. “Inapparent Infections and Cholera Dynamics.” *Nature* 454 (7206): 877–80. <https://doi.org/10.1038/nature07084>.
- King, Aaron A., Dao Nguyen, and Edward L. Ionides. 2016. “Statistical Inference for Partially Observed Markov Processes via the R Package **Pomp**.” *Journal of Statistical Software* 69 (12). <https://doi.org/10.18637/jss.v069.i12>.
- Korevaar, Hannah, C. Jessica Metcalf, and Bryan T. Grenfell. 2020. “Structure, Space and Size: Competing Drivers of Variation in Urban and Rural Measles Transmission.” *Journal of The Royal Society Interface* 17 (168): 20200010. <https://doi.org/10.1098/rsif.2020.0010>.
- Marino, J. A. Jr., S. D. Peacor, D. B. Bunnell, H. A. Vanderploeg, S. A. Pothoven, A. K. Elgin, J. R. Bence, J. Jiao, and E. L. Ionides. 2019. “Evaluating Consumptive and Non-consumptive Predator Effects on Prey Density Using Field Time-Series Data.” *Ecology* 100 (3): e02583. <https://doi.org/10.1002/ecy.2583>.
- Mietchen, Matthew S., Erin Clancey, Corrin McMichael, and Eric T. Lofgren. 2024. “Estimating SARS-CoV-2 Transmission Parameters Between Coinciding Outbreaks in a University

- Population and the Surrounding Community.” <https://doi.org/10.1101/2024.01.10.24301116>.
- Sun, Weizhe. 2024. “Model Based Inference of Stochastic Volatility via Iterated Filtering,” April.
- Tan, Kevin, Giles Hooker, and Edward L. Ionides. 2024. “Accelerated Inference for Partially Observed Markov Processes Using Automatic Differentiation.” arXiv. <https://doi.org/10.48550/arXiv.2407.03085>.
- Wen, L., Y. Yin, Q. Li, Z. Peng, and D. He. 2024. “Modeling the Co-Circulation of Influenza and COVID-19 in Hong Kong, China.” *Advances in Continuous and Discrete Models* 2024 (1): 1–9. <https://doi.org/10.1186/s13662-024-03830-7>.
- Wills, Adrian G., and Thomas B. Schön. 2021. “Stochastic Quasi-Newton with Line-Search Regularisation.” *Automatica* 127: 109503. <https://doi.org/10.1016/j.automatica.2021.109503>.